



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

Lara Hernandez Angel Husiel - 320060829

Laboratorio de Microcomputadoras

Grupo: 06 - Semestre: 2026-2

Practica 3:

Introducción a la plataforma Raspberry Pi Pico, GPIO como salida; programación en Micropython.

Profesor:

Ing. Moises Melendez Reyes

Fecha de Entrega:

15 de Marzo del 2026



1. Objetivo:

Conocer los recursos de la plataforma con el microcontrolador RP2040 contenida en la plataforma Raspberry Pi Pico; desarrollar algoritmos mediante la programación en Micropython utilizando el IDE Thonny para generar salidas a través de las terminales GPIO.

Actividad 1

Usando el IDE Thonny; escribir el siguiente código y comentar cada instrucción:

Propuesta de solución

Para lograr el parpadeo básico de un LED, se propone un script en MicroPython que manipula directamente los registros de hardware del microcontrolador. La solución requiere importar el módulo `machine` para tener acceso a la configuración de los pines físicos, y el módulo `utime` para generar bloqueos de tiempo en el procesador. Se configura el pin GPIO 0 como una salida digital. Posteriormente, mediante un bucle infinito `while True`, se alterna el estado lógico del pin enviando un nivel alto (1) y un nivel bajo (0), intercalando retardos de 500 milisegundos entre cada cambio de estado para hacer el parpadeo perceptible al ojo humano.

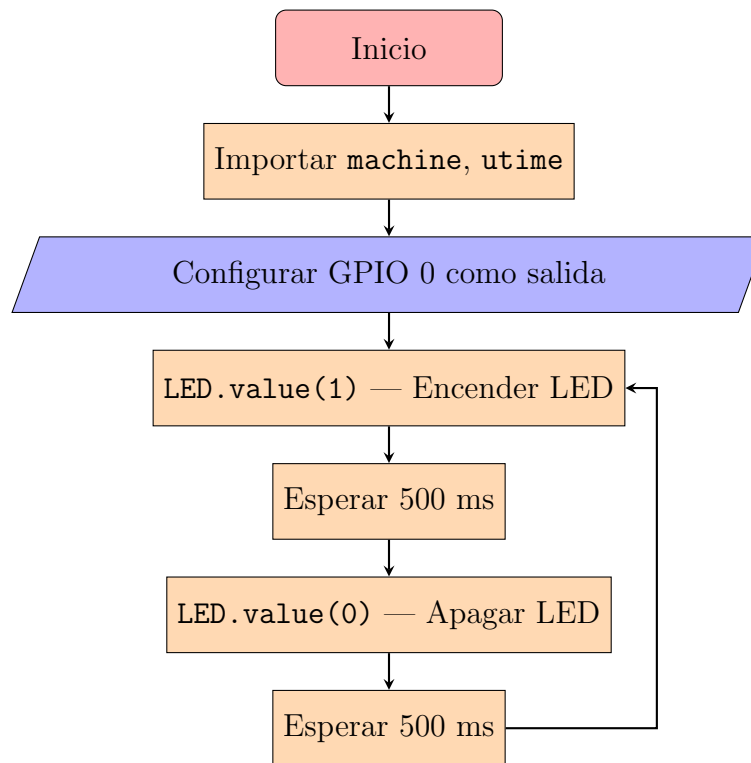


Diagrama de flujo — Actividad 1

El programa inicia importando las bibliotecas necesarias para el control de hardware (`machine`) y la gestión de temporización (`utime`). A continuación, se configura el pin GPIO 0 como una salida digital, lo que permite al microcontrolador enviar señales eléctricas a través de este terminal. El flujo entra entonces en un ciclo infinito donde se ejecutan cuatro operaciones secuenciales de forma ininterrumpida: primero se establece el nivel lógico alto (3.3 V) en el pin, encendiendo el LED; luego el procesador se bloquea durante 500 ms; después se establece el nivel lógico bajo (0 V), apagando el LED; y finalmente se ejecuta otro bloqueo de 500 ms. Al concluir estas cuatro instrucciones, el flujo retorna al inicio del ciclo, generando una señal cuadrada periódica de 1 Hz que se refleja como un parpadeo constante del LED.

Desarrollo

Listing 1: Código de la Actividad 1

```
1 import machine # Importa la librería para acceder al hardware de la
    Raspberry Pi Pico
2 import utime   # Importa la librería para manejar retardos de tiempo
```

```
(delays)

3
4 # Configura el pin GPIO 0 como una salida digital y lo asigna a la
   variable 'LED'
5 LED = machine.Pin(0, machine.Pin.OUT)
6
7 while True:                # Bucle infinito
8     LED.value(1)            # Enciende el LED (pone el pin GPIO 0 en
                               nivel alto '1')
9     utime.sleep_ms(500)      # Espera 500 milisegundos (medio segundo
                               )
10    LED.value(0)            # Apaga el LED (pone el pin GPIO 0 en
                               nivel bajo '0')
11    utime.sleep_ms(500)      # Espera 500 milisegundos
```

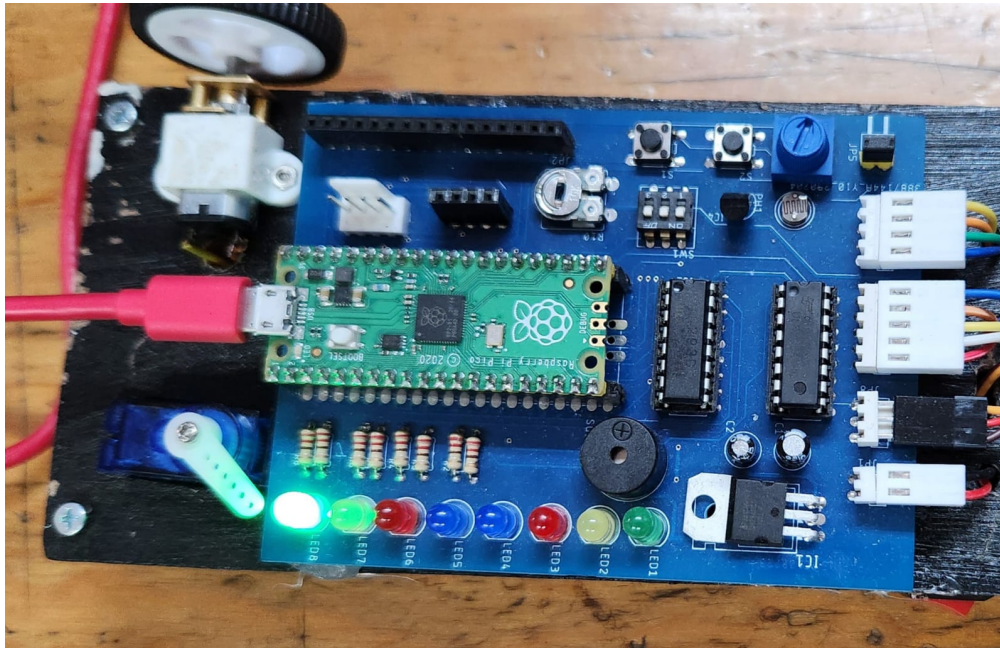


Figura 1: Estado de encendido (Nivel lógico alto). El microcontrolador Raspberry Pi Pico envía una señal de 3.3V a través del pin configurado como GPIO 0, lo que resulta en el encendido del indicador físico (LED8 color verde).

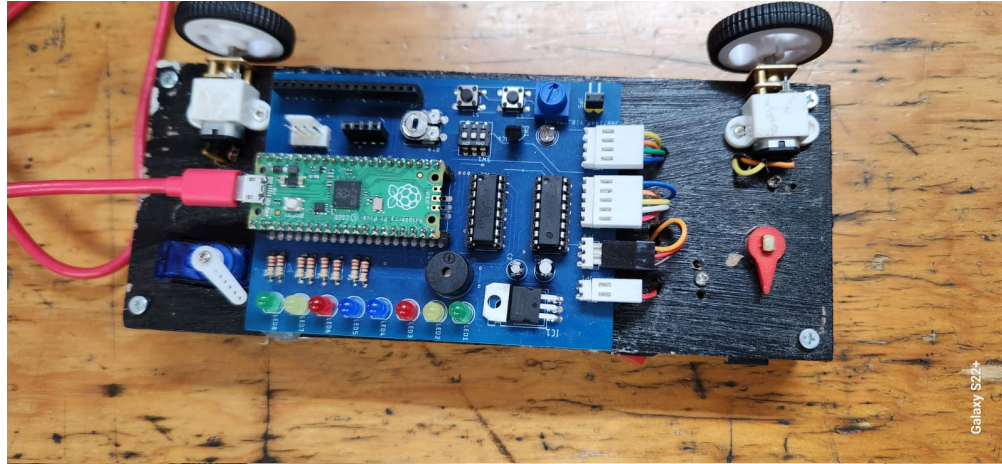


Figura 2: Estado de apagado (Nivel lógico bajo). Al cumplirse el retardo establecido de 500 milisegundos, la instrucción drena el voltaje del GPIO 0 hacia 0V, apagando el LED.

Análisis de resultados

La ejecución del script confirma el control directo sobre el hardware del microcontrolador. La instrucción `machine.Pin(0, machine.Pin.OUT)` configura correctamente el pin físico asociado al GPIO 0, estableciendo sus registros internos para operar como salida digital.

Al entrar en el ciclo `while True`, la instrucción `LED.value(1)` satura la salida, colocando un voltaje de 3.3V (nivel lógico alto) en el pin GPIO 0, lo que enciende el LED. Inmediatamente después, `utime.sleep_ms(500)` detiene la ejecución del procesador durante exactamente 500 milisegundos. Transcurrido este tiempo, `LED.value(0)` drena el voltaje a 0V (nivel lógico bajo), apagando el componente, seguido de otra pausa de 500 milisegundos. Este comportamiento secuencial y cíclico genera una onda cuadrada perfecta con un periodo total de 1000 milisegundos (1 segundo), logrando una frecuencia de parpadeo estable de 1 Hz.



Actividad 2

Realizar las modificaciones necesarias para que los tiempos sean más pequeños y más grandes que el código anterior.

Propuesta de solución

Para alterar la frecuencia de parpadeo dentro de un mismo ciclo de ejecución, se mantiene la configuración inicial del pin GPIO 0 como salida digital, pero se expande la lógica dentro del bucle principal. La propuesta divide el ciclo en dos fases: una de alta frecuencia y otra de baja frecuencia. Para los tiempos más pequeños, se establecen los cambios de estado del pin con retardos de 100 milisegundos. Para los tiempos más grandes, se añaden instrucciones idénticas de cambio de estado, pero acompañadas de retardos extendidos de 2000 milisegundos (2 segundos). Ambas fases se ejecutan secuencialmente dentro del bucle infinito.

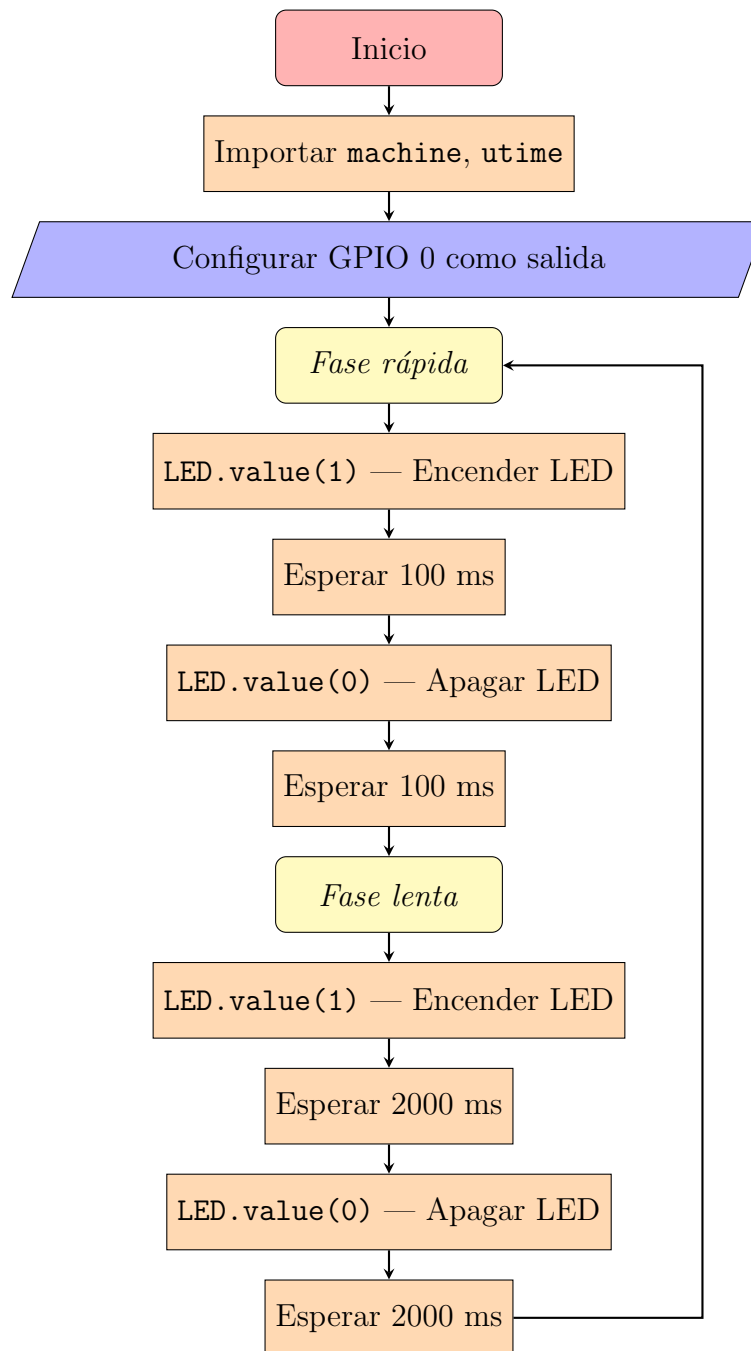


Diagrama de flujo — Actividad 2

El programa ingresa a un bucle infinito dividido en dos fases claramente diferenciadas. En la *fase rápida*, el pin alterna entre nivel alto y nivel bajo con retardos de apenas 100 ms, produciendo un parpadeo veloz con un periodo de 200 ms (5 Hz). Inmediatamente después,

sin interrumpir el flujo, se ejecuta la *fase lenta*, donde los mismos cambios de estado se acompañan de retardos de 2000 ms, generando un periodo de 4000 ms (0.25 Hz). Dado que ambas fases están concatenadas dentro del mismo `while True`, el flujo de datos es estrictamente secuencial: el microcontrolador completa la fase rápida antes de iniciar la lenta, y al finalizar la fase lenta, retorna al inicio del ciclo repitiendo la secuencia indefinidamente.

Desarrollo

Listing 2: Código de la Actividad 2

```
1 import machine # Importa la librería de hardware
2 import utime   # Importa la librería de tiempo
3
4 # Configura el pin GPIO 0 como salida
5 LED = machine.Pin(0, machine.Pin.OUT)
6
7 while True:      # Bucle principal
8     # --- Tiempos pequeños (Parpadeo rápido) ---
9     LED.value(1)  # Enciende el LED
10    utime.sleep_ms(100) # Espera solo 100 ms (muy rápido)
11    LED.value(0)   # Apaga el LED
12    utime.sleep_ms(100) # Espera 100 ms
13
14    # --- Tiempos grandes (Parpadeo lento) ---
15    LED.value(1)   # Enciende el LED
16    utime.sleep_ms(2000) # Espera 2000 ms (2 segundos)
17    LED.value(0)   # Apaga el LED
18    utime.sleep_ms(2000) # Espera 2 segundos
```

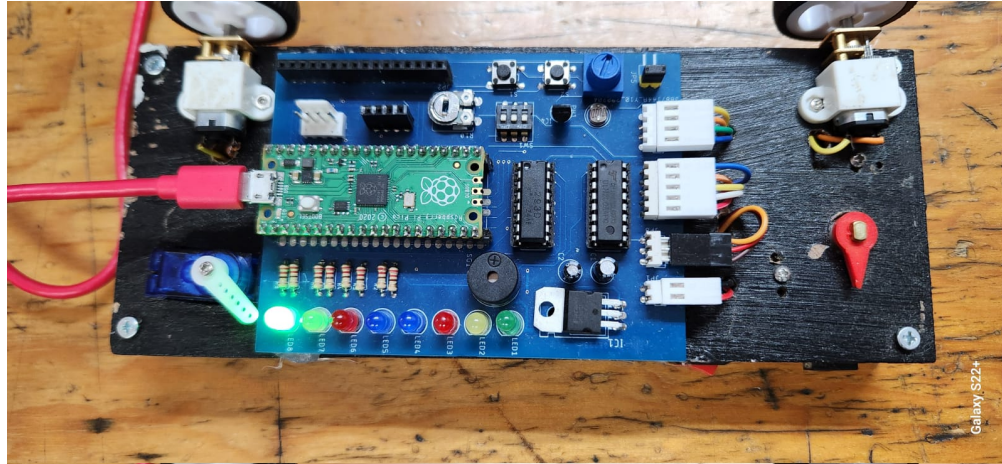



Figura 3: Fase de activación durante el ciclo asimétrico. El pin GPIO 0 se encuentra saturado a nivel lógico alto, encendiendo el LED durante las pausas cortas (100 ms) y largas (2000 ms).

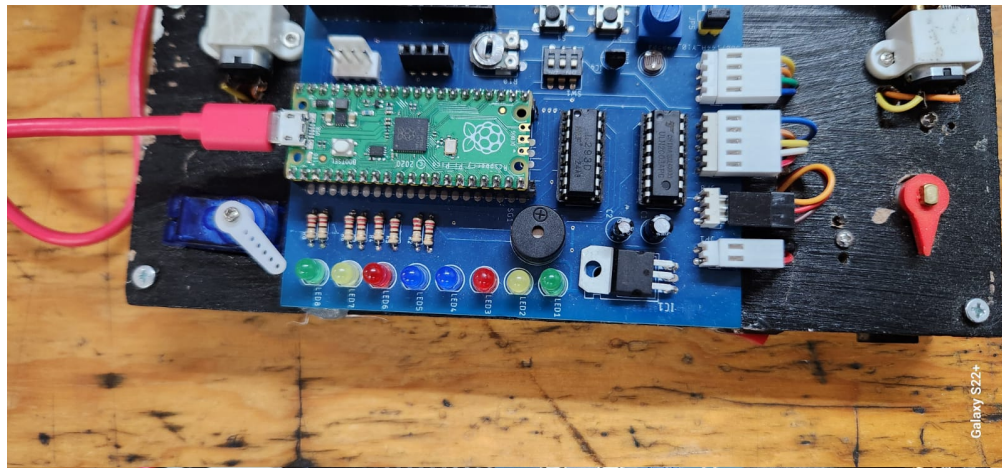


Figura 4: Fase de desactivación. El pin GPIO 0 corta el suministro de voltaje al indicador visual, respetando los bloqueos de tiempo instruidos al procesador antes de reiniciar el bucle principal.

Análisis de resultados

La modificación de los parámetros en la función `utime.sleep_ms()` evidencia la naturaleza bloqueante de esta instrucción sobre el hilo de ejecución principal del microcontrolador. Durante la primera fase del bucle, los retardos de 100 milisegundos fuerzan al pin GPIO 0



a conmutar rápidamente entre 3.3V y 0V. Esto genera un tren de pulsos muy corto con un periodo de 200 milisegundos, visualizándose físicamente como un parpadeo o estrobo rápido.

Posteriormente, el flujo del programa avanza a la segunda fase, donde las instrucciones `utime.sleep_ms(2000)` obligan al procesador a mantener el estado lógico actual del pin durante 2 segundos completos antes de ejecutar la siguiente línea de código. Esto crea un periodo de 4000 milisegundos (4 segundos), resultando en un encendido y apagado prolongado. La combinación de ambas rutinas en el mismo `while True` demuestra que los tiempos de retardo dictan absolutamente la anchura de los pulsos generados, permitiendo crear secuencias asimétricas de temporización sobre el mismo componente de hardware sin necesidad de usar temporizadores (*timers*) por interrupción.

Actividad 3

Realizar las modificaciones requeridas para que el efecto se muestre en otro GPIO (podrá usar GPIO1 al GPIO 7); usar retardos a conveniencia.

Propuesta de solución

La solución a este requerimiento implica una reasignación del hardware. En lugar de inicializar el objeto pin en el GPIO 0, se modifica el constructor para apuntar al GPIO 5, el cual está físicamente conectado a un LED rojo según las especificaciones del circuito. La variable controladora se renombra a `LED_ROJO` para mantener claridad en el código. Adicionalmente, se ajusta la función de retardo a un valor intermedio de conveniencia, fijando la pausa en 300 milisegundos tanto para el estado alto como para el bajo.

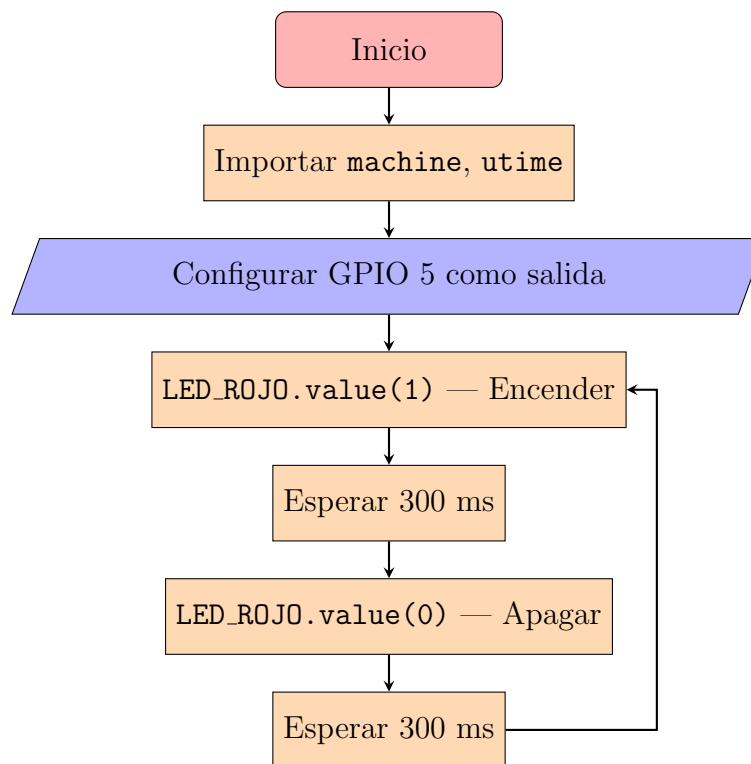


Diagrama de flujo — Actividad 3

Este programa sigue una estructura análoga a la Actividad 1, con la diferencia fundamental en el canal de salida utilizado. Tras importar las bibliotecas, se instancia el pin GPIO 5 como

salida digital, redirigiendo las señales de control hacia un terminal físico distinto donde se conecta un LED rojo. El bucle infinito ejecuta la misma lógica de conmutación: se establece un nivel alto (3.3 V) para encender el LED, se pausa 300 ms, se establece un nivel bajo (0 V) para apagarlo, y se pausa otros 300 ms. La flecha de retorno en el diagrama indica que el flujo regresa al inicio del ciclo de forma perpetua, generando una señal cuadrada con un periodo de 600 ms (≈ 1.67 Hz). Este diagrama demuestra que el algoritmo es completamente modular: el cambio de un solo parámetro en la configuración inicial (el número de GPIO) es suficiente para trasladar toda la secuencia de operaciones a un pin diferente del microcontrolador.

Desarrollo

Listing 3: Código de la Actividad 3

```
1 import machine # Importa la librería de hardware
2 import utime   # Importa la librería de tiempo
3
4 # Configura el pin GPIO 5 (LED ROJO según tabla) como salida
5 LED_ROJO = machine.Pin(5, machine.Pin.OUT)
6
7 while True:           # Bucle infinito
8     LED_ROJO.value(1) # Enciende el LED conectado al GPIO 5
9     utime.sleep_ms(300) # Retardo a conveniencia (300 ms)
10    LED_ROJO.value(0)  # Apaga el LED conectado al GPIO 5
11    utime.sleep_ms(300) # Retardo a conveniencia (300 ms)
```

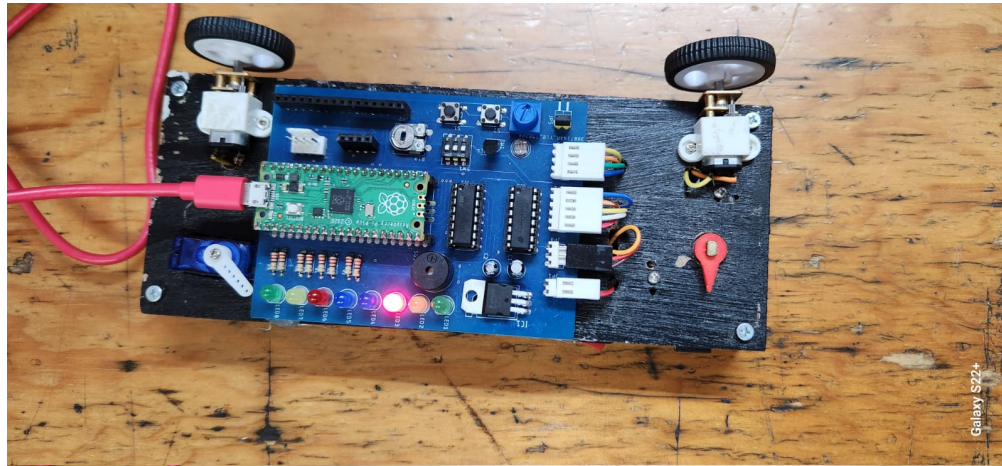



Figura 5: Estado de encendido (Nivel lógico alto). El microcontrolador redirige la señal de 3.3V hacia el pin configurado como GPIO 5, encendiendo exitosamente el indicador físico correspondiente (LED color rojo).

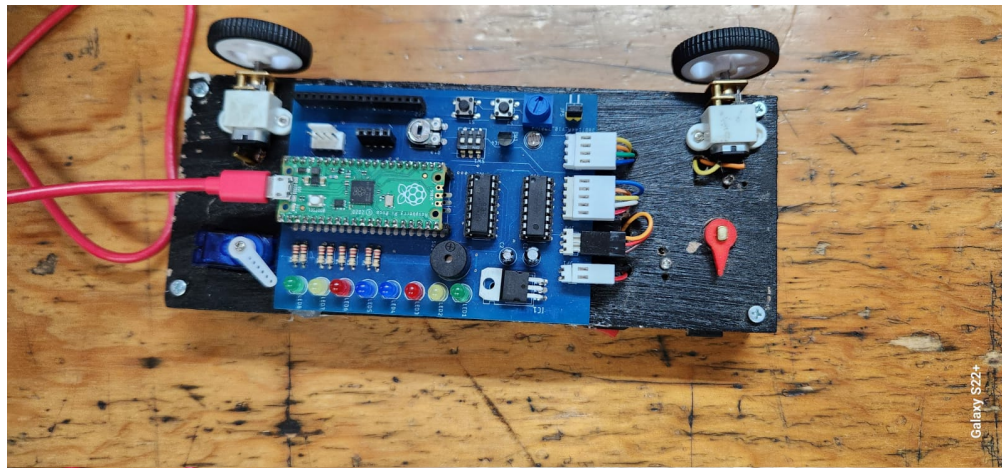


Figura 6: Estado de apagado (Nivel lógico bajo). Al transcurrir el retardo asignado de 300 milisegundos, el procesador corta el suministro de voltaje en el terminal GPIO 5, desactivando el componente visual.

Análisis de resultados

El cambio más significativo en la ejecución radica en el mapeo de los periféricos internos del microcontrolador. Al modificar el primer argumento de la clase a `machine.Pin(5, machine.Pin.OUT)`, el compilador de MicroPython redirige las señales de control digital ha-



cia el pin físico correspondiente al GPIO 5, dejando inactivo el GPIO 0 utilizado en las actividades previas.

La lógica del bucle infinito permanece estructuralmente idéntica, conmutando los niveles de voltaje por medio de `LED_ROJO.value(1)` y `LED_ROJO.value(0)`. El ajuste del retardo a `utime.sleep_ms(300)` genera una onda cuadrada constante con un periodo de 600 milisegundos. Este resultado comprueba que el algoritmo de parpadeo es modular e independiente del pin seleccionado; basta con instanciar correctamente el canal de salida al inicio del programa para que cualquier terminal GPIO del microcontrolador ejecute la secuencia de señales eléctricas programada.

Actividad 4

Realizar un programa que genere las siguientes salidas:

GPIO2	GPIO3
1	0
0	1

@85 ms entre estados

Tabla 3-3. Control; actividad 4

Propuesta de solución

La propuesta de solución para implemen lo solicitado, consiste en programar dos terminales de salida digital del microcontrolador Raspberry Pi Pico para que tengan comportamientos opuestos, pues a diferencia de otras actividades donde se requiere pura sincronización aquí cuando el GPIO 2 se encuentra en estado lógico alto (3.3 V), el GPIO 3 debe permanecer en estado bajo (0 V), e invertirse inmediatamente después de un intervalo de tiempo. Se ha configurado un retardo de 85 ms utilizando la librería `utime`, lo que genera una frecuencia de oscilación de aproximadamente 5.88 Hz, produciendo un efecto visual de parpadeo alternado entre los LEDs de la placa.

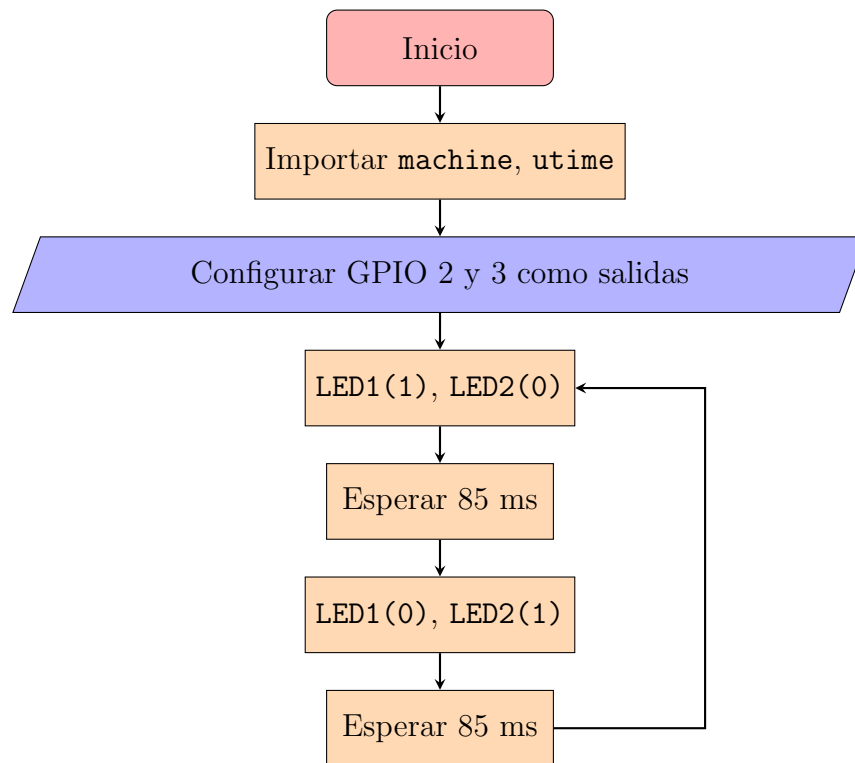


Diagrama de flujo — Actividad 4

Desarrollo

Listing 4: Código de la Actividad 4

```
1 import machine
2 import utime
3
4 # Configura los pines GPIO 2 y GPIO 3 como salidas
5 LED1 = machine.Pin(2, machine.Pin.OUT)
6 LED2 = machine.Pin(3, machine.Pin.OUT)
7
8 while True:
9     # Estado 1: GPIO2 en Alto (1), GPIO3 en Bajo (0)
10    LED1.value(1)
11    LED2.value(0)
12    utime.sleep_ms(85) # Espera 85 ms entre estados
13
```

```
14 # Estado 2: GPIO2 en Bajo (0), GPIO3 en Alto (1)
15 LED1.value(0)
16 LED2.value(1)
17 utime.sleep_ms(85) # Espera 85 ms entre estados
```

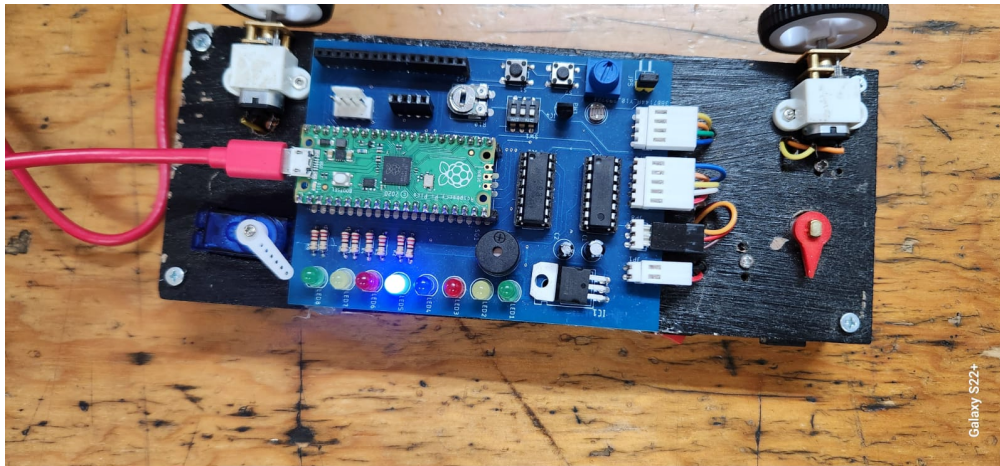


Figura 7: Estado 1 de la secuencia. El microcontrolador establece el GPIO 2 en alto por lo que el primer LED se enciende y el segundo permanece apagado por estar en el nivel lógico de bajo.

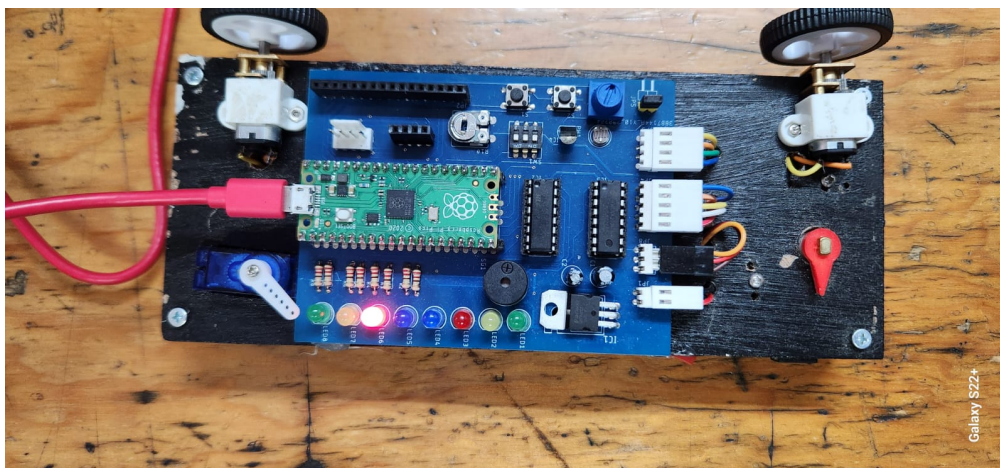


Figura 8: Estado 2 de la secuencia. Tras el retardo de 85 ms, se invierte la polaridad lógica y el GPIO 3 se activa y el GPIO 2 se apaga, completando el ciclo de alternancia.

Análisis de resultados

Como se vió en las Figura 7 y 8 se cumplió con lo solicitado en la tabla de verdad ya que al ejecutarse el bucle infinito, el procesador alterna los estados de los pines de manera cíclica, lo cual se evidencia visualmente al notar que solo un led se mantiene encendido en un cierto instante, mientras que el otro permanece apagado.

La implementación de `utime.sleep_ms(85)` fue vital para poder apreciar de mejor manera los estados del sistema ya que un tiempo menor podría causar un solapamiento visual debido a la persistencia retiniana, mientras que un tiempo mayor ralentizaría la respuesta. Este ejercicio nos demuestra la capacidad de la Raspberry Pi Pico para gestionar procesos de conmutación rápida y coordinada en el control de la sincronización de periféricos.

Actividad 5

Realizar un programa que prenda y apague las GPIO0 al GPIO7; el circuito corresponde en su totalidad al mostrado en la figura 3.3, usar los retardos a elección.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

Tabla 3-4. Control; actividad 5

Propuesta de solución

Para esta actividad número 5 se escala el uso del hardware al gestionar un bus completo de 8 bits (GPIO 0 a GPIO 7), es por ello que en lugar de instanciar cada pin de forma individual, se propone el uso de una ****lista de objetos**** y un ciclo **for** para la inicialización por lo que se consigue optimizar el uso de memoria y la legibilidad del código. La lógica de control aplica un estado uniforme a todo el conjunto: se recorre la lista para establecer un nivel alto (3.3 V) en todos los terminales simultáneamente, se mantiene por 500 ms, y posteriormente se repite el proceso para el nivel bajo (0 V), Con esto se busca demostrar la flexibilidad de MicroPython para manejar múltiples periféricos de forma colectiva.

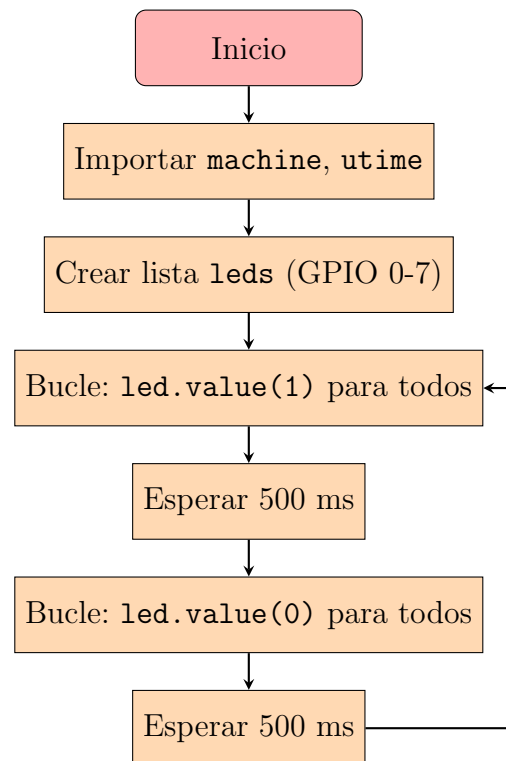


Diagrama de flujo — Actividad 5

Desarrollo

Listing 5: Código de la Actividad 5

```
1 import machine
2 import utime
3
4 # Crear una lista con los pines del 0 al 7 configurados como salida
5 # Esto facilita encenderlos y apagarlos todos juntos con un bucle
6 leds = [machine.Pin(i, machine.Pin.OUT) for i in range(8)]
7
8 while True:
9     # --- Estado: Todos en '1' (Encendidos) ---
10    for led in leds:
11        led.value(1)          # Asigna el valor 1 a cada LED de la
12                               lista
13    utime.sleep_ms(500)       # Retardo a elección (500 ms)
```

```
13
14     # --- Estado: Todos en '0' (Apagados) ---
15     for led in leds:
16         led.value(0)          # Asigna el valor 0 a cada LED de la
                                lista
17     utime.sleep_ms(500)       # Retardo a elección (500 ms)
```

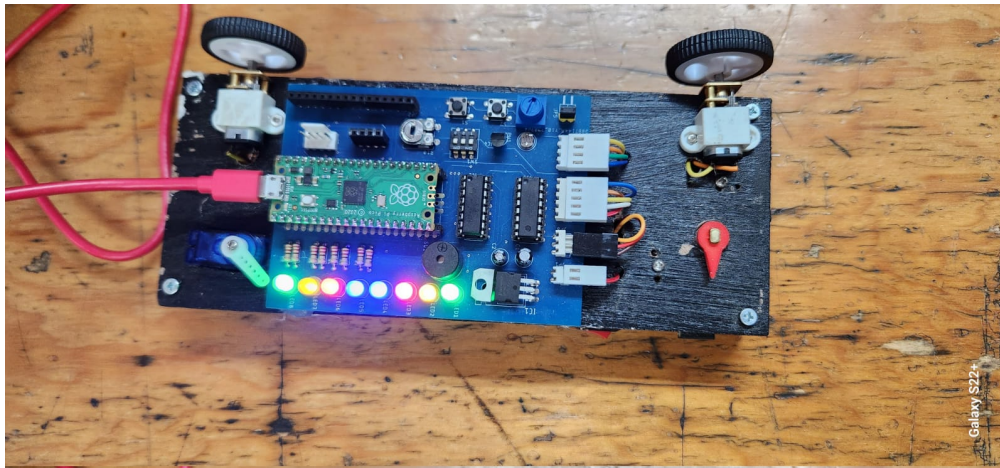


Figura 9: Estado de encendido, aquí se observa la activación simultánea de la barra de 8 LEDs (GPIO 0-7), validando que el microcontrolador suministra corriente correctamente a todo el bus configurado.

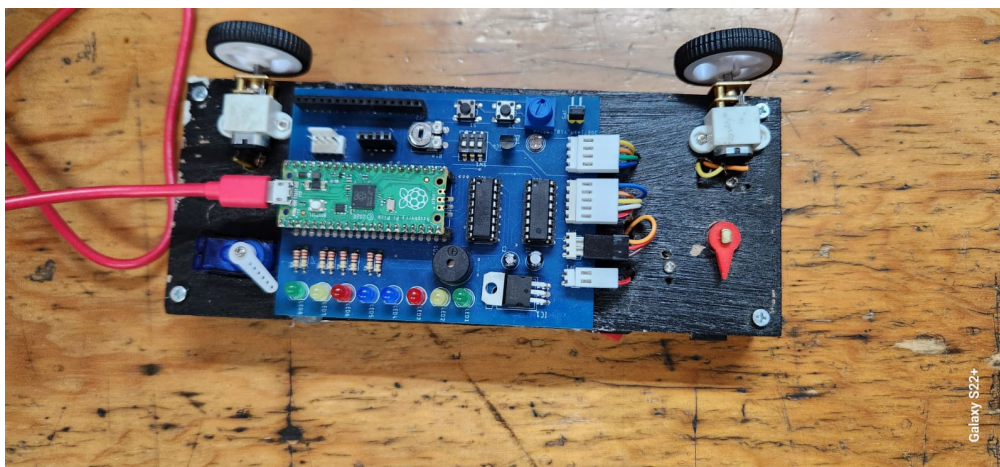


Figura 10: Estado de apagado. Tras el retardo de 500 ms, el procesador corta el flujo en todos los terminales, resultando en la desactivación completa de los indicadores visuales.



Análisis de resultados

El comportamiento observado en las figuras 9 y 10 nos confirma la eficacia de la programación mediante colecciones ya que al ejecutar el código, la placa responde encendiendo simultáneamente los 8 leds al recorrer la lista y cambiar el estado de los pines es despreciable frente al retardo de 500ms, dando la impresión de una activación instantánea. Este resultado es relevante porque valida la integridad de las conexiones físicas desde el **GPIO 0** hasta el **GPIO 7** en la placa de expansión y el uso de la instrucción `range(8)` asegura que no existan pines omitidos en la secuencia. Este método de control ^{en bloque} es fundamental para futuras aplicaciones más avanzadas.

Actividad 6

Repetir la acción anterior 10 veces; mantener apagado 2 segundos y repetir acción.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7	
0	0	0	0	0	0	0	0	10 repeticiones (200 ms)
1	1	1	1	1	1	1	1	
0	0	0	0	0	0	0	0	
1	1	1	1	1	1	1	1	
0	0	0	0	0	0	0	0	2 segundos

Tabla 3-5. Control; actividad 6

Propuesta de solución

Para esta actividad se implementa una estructura de control anidada que permite gestionar ciclos de repetición finitos dentro de un bucle infinito tomando como base el código realizado en la actividad anterior. La solución utiliza un objeto de lista para los pines **GPIO 0** al **7** y el flujo principal consiste en un bucle `for` que ejecuta 10 iteraciones del parpadeo de todos los leds (200 ms por estado) y una vez completada esta secuencia, el programa sale del bucle interno para ejecutar una pausa de 2000ms (2 segundos) con todos los terminales en nivel lógico bajo (0 V), antes de reiniciar el ciclo completo.

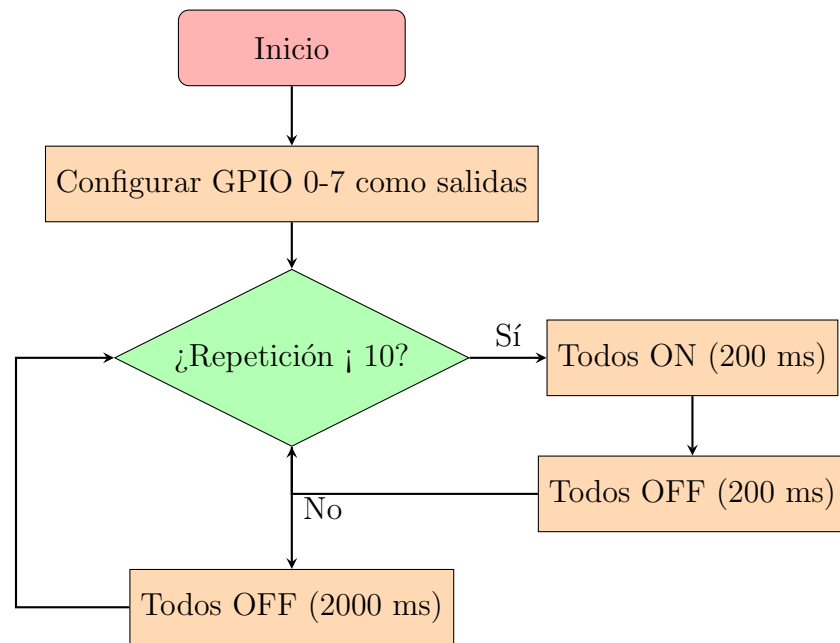


Diagrama de flujo — Actividad 6

Desarrollo

Listing 6: Código de la Actividad 6

```
1 import machine
2 import utime
3
4 # Lista de pines configurados como salidas (GPIO 0 a 7)
5 leds = [machine.Pin(i, machine.Pin.OUT) for i in range(8)]
6
7 while True:
8     # Repetir la acción de parpadeo 10 veces
9     for repeticion in range(10):
10         # Todos encendidos
11         for led in leds:
12             led.value(1)
13         utime.sleep_ms(200) # Retardo de 200 ms (según tabla)
14
15         # Todos apagados
16         for led in leds:
```



```
17         led.value(0)
18         utime.sleep_ms(200) # Retardo de 200 ms (según tabla)
19
20     # Mantener apagado 2 segundos después de las 10 repeticiones
21     for led in leds:
22         led.value(0)          # Asegurar que todos estén apagados
23         utime.sleep_ms(2000)  # Retardo de 2000 ms (2 segundos)
```

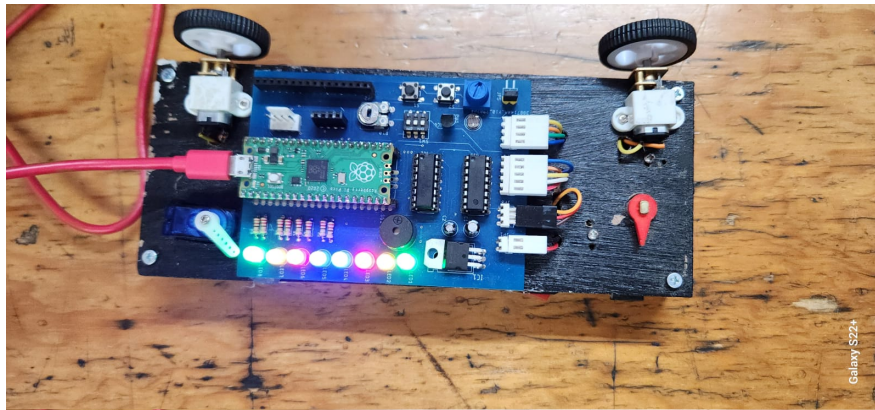


Figura 11: Estado de encendido durante los 10 ciclos de encendido y apagado.

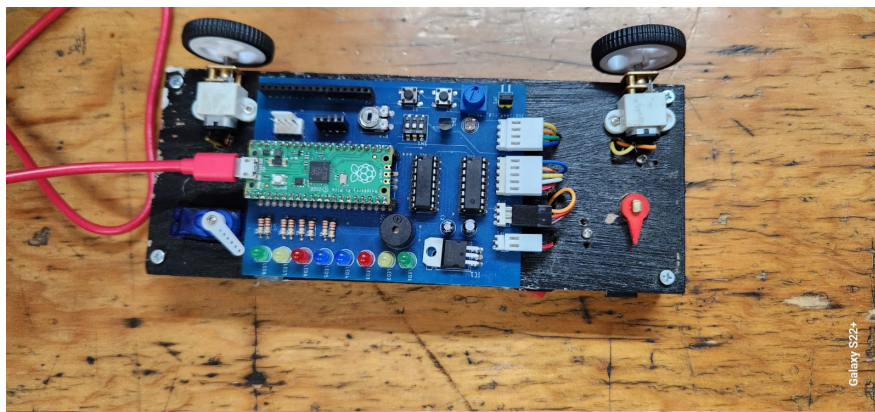


Figura 12: Captura del estado de apagado por 2 s. Los pines GPIO 0-7 se mantienen en 0 V tras terminar las 10 iteraciones de encendido y apagado.

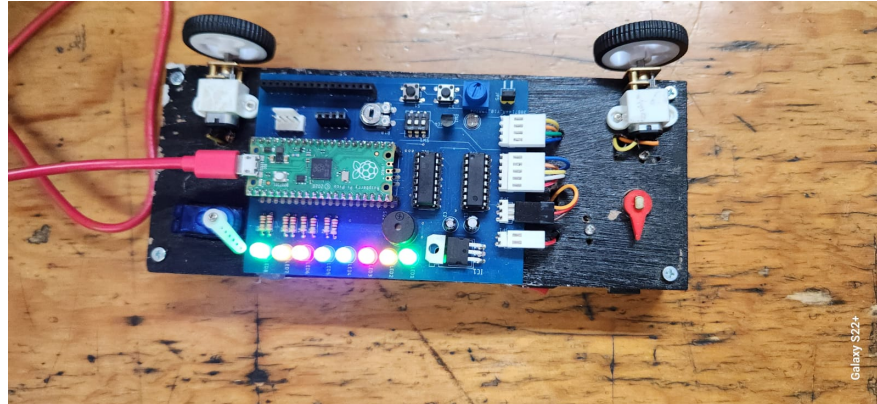


Figura 13: Retorno al estado de encendido tras los 2 segundos de apagado. La barra de LEDs vuelve a mostrar una iluminación uniforme

Análisis de resultados

Las imágenes capturadas en la Figura 11, 12 y 13 reflejan los dos estados críticos del sistema: la activación total del bus de 8 bits y el estado de reposo.

Mediante la observación del sistema, se confirmó que el microcontrolador realiza exactamente 10 destellos rápidos antes de entrar en el estado de espera prolongada y la transición entre el bucle `for repeticion in range(10)` y el retardo de `utime.sleep_ms(2000)` ocurrió conforme a lo esperado en el procesamiento, demostrando que el hardware del Raspberry Pi Pico maneja eficientemente estructuras de iteración de Python.

Actividad 7

Usando los GPIO0 al GPIO7; realizar un programa que genere la siguiente secuencia; usar retardos de 100ms.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1

Tabla 3-6. Control; actividad 7

Propuesta de solución

En esta septima actividad se nos solicitó realizar una especie de corrimiento de bits a la derecha con los leds de las Raspberry por lo que se implementó un algoritmo de desplazamiento lineal de un único bit activo a lo largo de un bus de 8 bits par lograr implementar la tabla de verdad solicitada, se utiliza una estructura de bucles anidados: un bucle principal recorre los índices de la lista de LEDs (0 al 7), mientras que un bucle interno se encarga de "limpiar" el estado de todos los terminales antes de activar el pin correspondiente al índice actual. Este método garantiza que, en cualquier instante de tiempo dado, solo un LED se encuentre bajo un nivel lógico alto (3.3V) y el uso de un retardo de 100 ms genera un efecto visual de traslación fluida de izquierda a derecha en la barra de indicadores físicos.

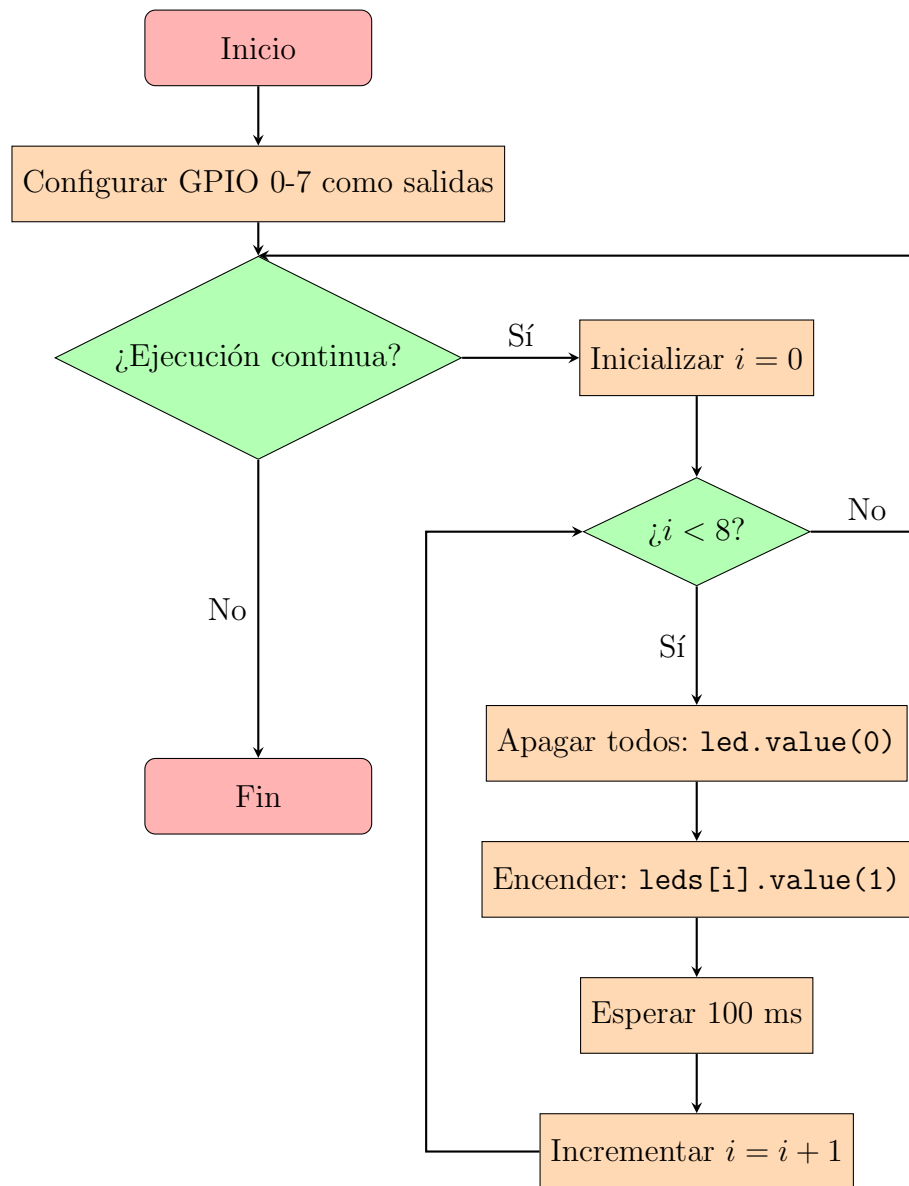


Figura 14: Diagrama de flujo — Actividad 7 (Efecto de luz recorriente)

Desarrollo

Listing 7: Código de la Actividad 7

```

1 import machine
2 import utime
3
4 # Lista de pines configurados como salidas

```

```
5 leds = [machine.Pin(i, machine.Pin.OUT) for i in range(8)]
6
7 while True:
8     # Recorrer cada LED de la lista (del GPIO 0 al 7)
9     for i in range(8):
10        # Apagamos todos los LEDs primero para limpiar el estado
11        for led in leds:
12            led.value(0)
13
14        # Encendemos solo el LED actual en la secuencia
15        leds[i].value(1)
16
17        # Retardo de 100 ms según la solicitud
18        utime.sleep_ms(100)
```

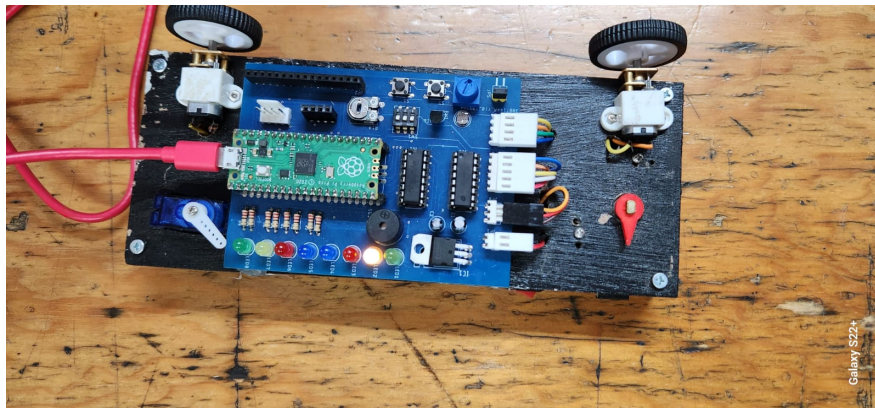


Figura 15: Encendido del GPIO7 y apagado de los demás.

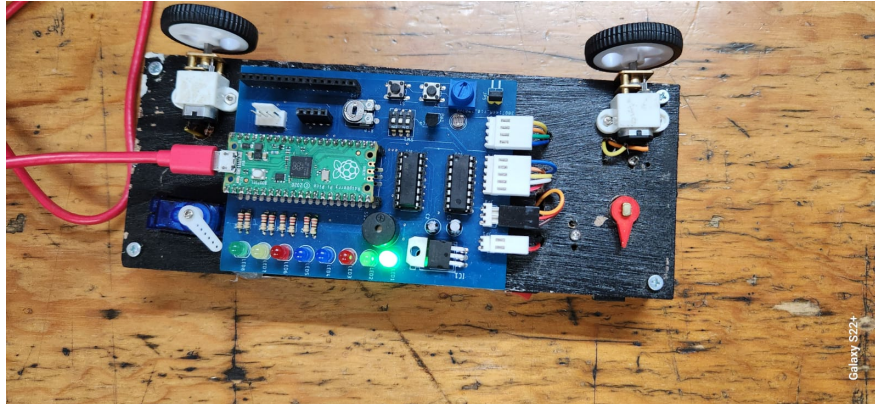


Figura 16: Encendido del GPIO8 y apagado de los demás, ultimo estado de la tabla.

Análisis de resultados

El análisis de la ejecución física tal y como se muestra en la Figura 14 y 15 confirma que el programa cumple estrictamente con la secuencia definida en la Tabla 3-6. Al utilizar una lista indexada, el microcontrolador puede direccionar de forma dinámica cada terminal GPIO, permitiendo que la señal alta "viaje" desde el GPIO 0 hasta el GPIO 7 de manera sucesiva.

En este ejercicio se visualiza una especie de corrimiento de bits a una frecuencia de (10 Hz) que es ideal para observar el desplazamiento individual de cada LED sin que la persistencia visual mezcle los estados.

Actividad 8

Usando los GPIO0 al GPIO7; realizar un programa que genere la siguiente secuencia; usar retardos de 100ms.

Propuesta de solución

Al igual que en actividades anteriores dado que se requiere interactuar con el hardware, en este caso los pines GPIO así como gestionar pausas en la ejecución del programa es necesario importar las librerías `machine` y `utime`.

Como en la actividad se solicita usar de los GPIO0 al GPIO7 es necesario realizar la configuración de cada uno de los GPIO, sin embargo esto se puede simplificar por recorrien-

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	0	0	0
1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1

Tabla 3-7. Control; actividad 8

do los números de del 0 al 7, donde en cada iteración (*i*) se crea un objeto del tipo `machine.Pin(i,machine.Pin.OUT)` donde estos objetos se guardaran en una lista llamada `leds` de forma que para interactuar con los pines GPIO se debe acceder al elemento correspondiente de la lista `leds` teniendo así un mejor control y organización de los pines GPIO.

Dado que queremos que la secuencia de encendido de leds se repita de forma indefinida, se debe colocar la lógica de la secuencia dentro de un bucle `while` infinito, para que este se encuentre repitiéndose aca vez que finalice la secuencia. De acuerdo a la tabla de la actividad podemos ver que la secuencia consiste es que en cada paso se va encendido un led adicional, es decir en la primera iteración se enciende el primer led, en la segunda iteración el primer y segundo led, así hasta los 8 leds.

Para lograr las secuencia descrita, se debe de implementar un ciclo iterativo de 8 ciclos, con la finalidad que en cada ciclo se encienda el led correspondiente, dicho ciclo iterativo se implementa con un `for`, donde en cada iteración enciende el led correspondiente, como se encuentra en la lista de pines que anteriormente se definió simplemente se debe acceder al indice de la lista que coincide con la iteración y asignarles el valor de 1 para que asi se encienda el led, antes de pasar a la siguiente iteración es necesario poner una pausa, para que sea visible el encendido de los leds, en este caso de indico que la pausa debe ser de 100[ms], de esta forma se consigue la secuencia descrita en la tabla.

Por último como se debe repetir la secuencia, es necesaria que una vez que se enciendan

todos los leds, se deben volver a apagar todos para que Posteriormente se vuelva a ejecutar la secuencia de encendido y sea visible el cambio de estado de apagado a encendido.

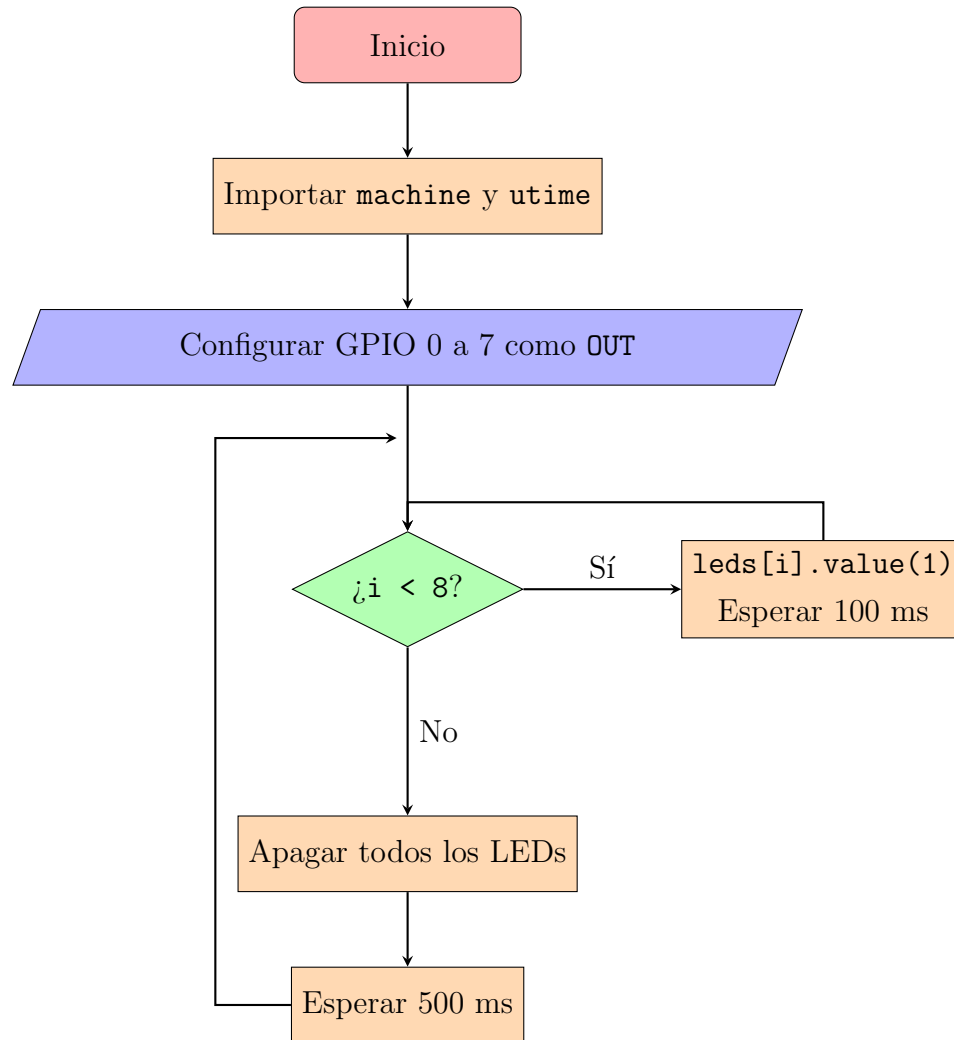


Diagrama de flujo: Secuencia de llenado GPIO0-GPIO7

Desarrollo

Listing 8: Código de la Actividad 8

```
1 import machine
2 import utime
3
4 # Lista de pines configurados como salidas
```

```
5 leds = [machine.Pin(i, machine.Pin.OUT) for i in range(8)]
6
7 while True:
8     # --- Secuencia de llenado ---
9     for i in range(8):
10         leds[i].value(1)      # Enciende el LED actual, dejando los
                                # anteriores encendidos
11         utime.sleep_ms(100)   # Retardo de 100 ms
12
13     # Apagamos todos para reiniciar el efecto visual
14     for led in leds:
15         led.value(0)
16     utime.sleep_ms(500)      # Pausa antes de repetir
```

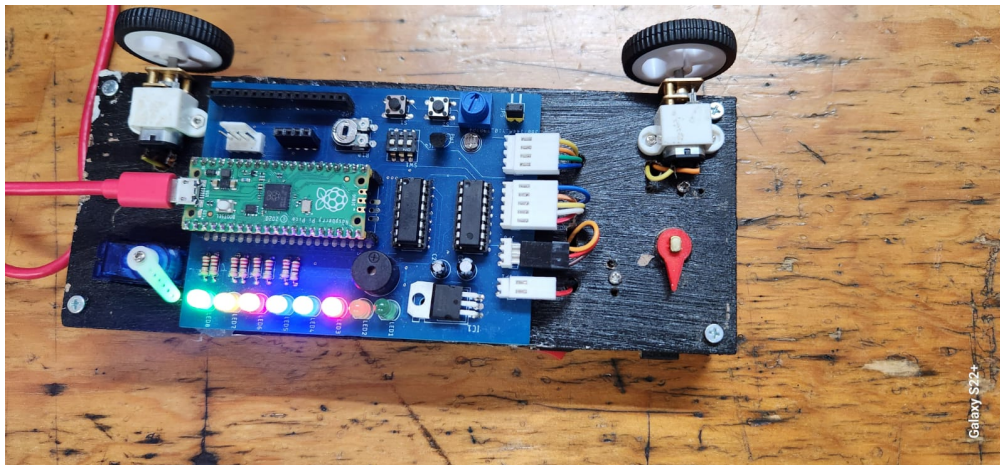


Figura 17: Estado de encendido de los GPIO0-GPIO5 Iteración número 6

Análisis de resultados

En la imagen se puede observar el correcto funcionamiento de la secuencia descrita en la actividad justo en la iteración número 6. Como se describe por cada iteración se debe de encender un nuevo led manteniendo los anteriores. De forma que la imagen corresponde con la iteración descrita

- Iteración 1: Encendido de los leds correspondientes a los pines GPIO0
- Iteración 2: Encendido de los leds correspondientes a los pines GPIO0-GPIO1

- Iteración 3: Encendido de los leds correspondientes a los pines GPIO0-GPIO2
- Iteración 4: Encendido de los leds correspondientes a los pines GPIO0-GPIO3
- Iteración 5: Encendido de los leds correspondientes a los pines GPIO0-GPIO4
- Iteración 6: Encendido de los leds correspondientes a los pines GPIO0-GPIO5
- Iteración 7: Encendido de los leds correspondientes a los pines GPIO0-GPIO6
- Iteración 8: Encendido de los leds correspondientes a los pines GPIO0-GPIO7

Coincidiendo los leds encendido con los encendido de los pines de GPIO0-GPIO5, por lo que se implemento de forma correcta la secuencia descrita.

Actividad 9

Usando los GPIO0 al GPIO7; realizar un programa que genere un contador de 8 bits de forma ascendente; usar retardos de 50ms.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	0
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1

Tabla 3-8. Control; actividad 9

Propuesta de solución

La metodología es la misma dado que se van a ocupar 8 pines GPIO donde cada uno corresponde a un led, y cada led representa un led, de forma que al ser necesario interactuar el hardware así como gestionar retardos se importan las librerías `machine` y `utime`. Se genera con la misma lógica que en el ejercicio anterior una lista de objetos del tipo `machine.Pin`



con la finalidad de tener en una sola estructura todos los pines, sin embargo dado que se va a realizar un contador, es importante destacar que el elemento `led[0]` es el bit menos significativo (correspondiente al LED 8 en la Raspberry) mientras que el elemento `led[7]` es el bit más significativo (correspondiente al LED 1 en la Raspberry).

Dado que se busca que el contador se reinicie una vez llegue a su valor máximo y este vuelva a comenzar desde cero, se debe colocar la lógica del contador dentro de un bucle infinito. Para el bucle que corresponde al contador se debe indicar un ciclo que vaya desde 0 a 255 ya que esos los números que se pueden representar con 8 bits, de forma que cuando el contador llegue a 255 se reinicie el contador.

Para cada iteración del contador es necesario indicar que leds se deben encender para representar el número que corresponde al contador. Para conocer que leds se deben de encender se realizan operaciones matemáticas a nivel bits, donde se itera el valor de cada led, donde considerando el valor del contador se realiza un desplazamiento a la derecha de acuerdo al valor del bit, con la finalidad de llevar el bit de interés a la posición 0 para posteriormente realizar una operación AND con el valor de 1 garantizando que el resultado solo pueda ser 0 o 1. Una vez que obtenemos el estado del bit correspondiente se le asigna el valor correspondiente a su pin, se esperan 50ms y se vuelve a incrementar el contador.

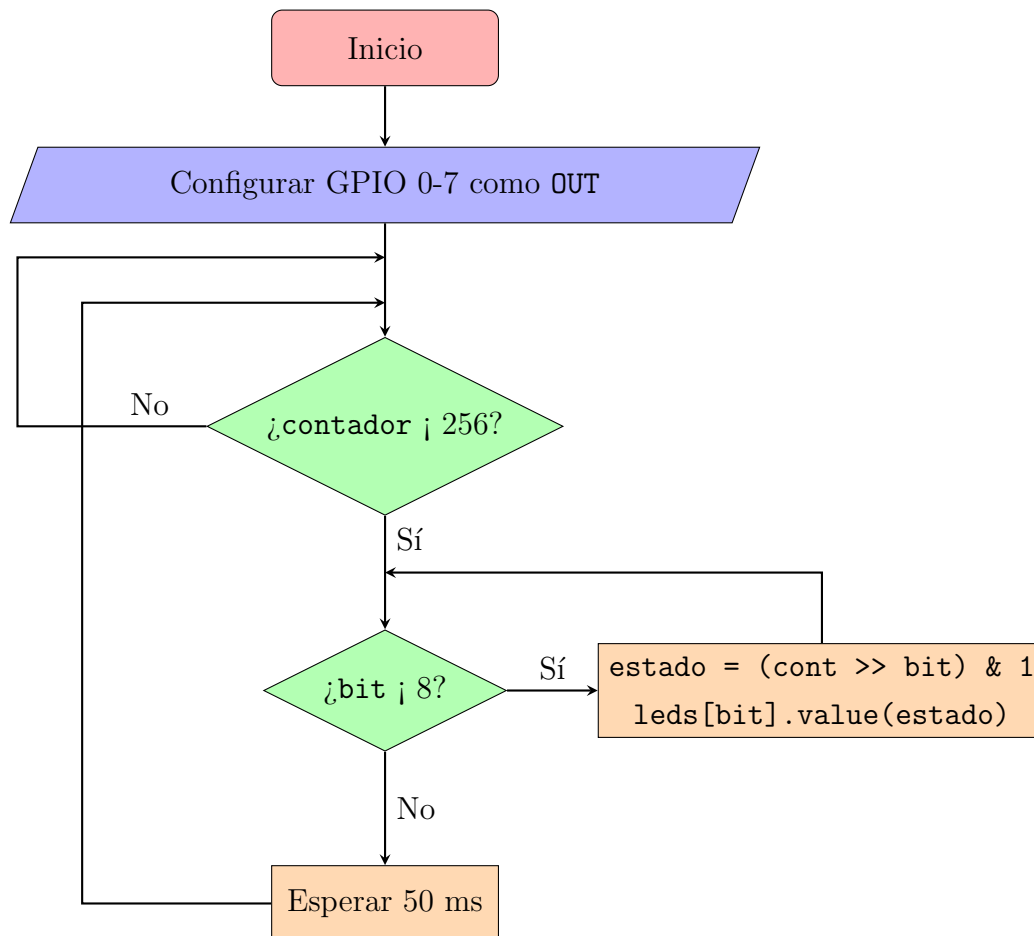


Diagrama de flujo: Contador 8-bits

Desarrollo

Listing 9: Código de la Actividad 9

```
1 import machine
2 import utime
3
4 # Lista de pines configurados como salidas
5 leds = [machine.Pin(i, machine.Pin.OUT) for i in range(8)]
6
7 while True:
8     # Contar de 0 a 255 (8 bits)
9     for contador in range(256):
```

```
10     # Recorrer los 8 bits del número actual
11     for bit in range(8):
12         # Extraemos el valor del bit en la posición actual (0 o
13         # 1)
14         # Desplazamos el contador a la derecha 'bit' veces y
15         # hacemos AND con 1
16         estado_bit = (contador >> bit) & 1
17
18         # Asignamos ese estado al LED correspondiente
19         leds[bit].value(estado_bit)
20
21     # Retardo de 50 ms entre cada número
22     utime.sleep_ms(50)
```

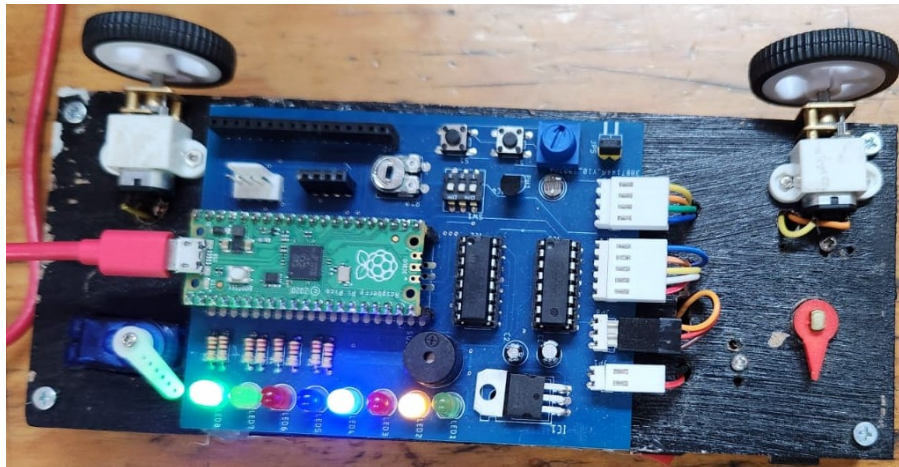


Figura 18: Actividad 9: Contador = 81 - 01010001

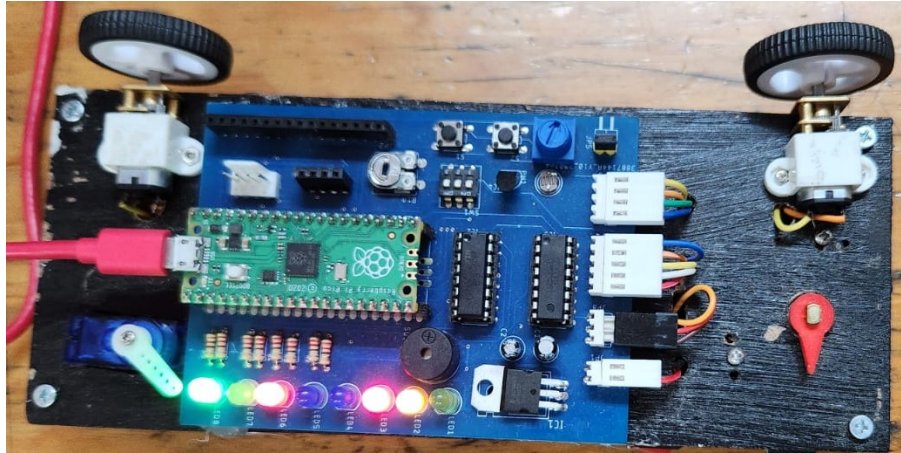


Figura 19: Actividad 9: Contador = 101 - 01100101

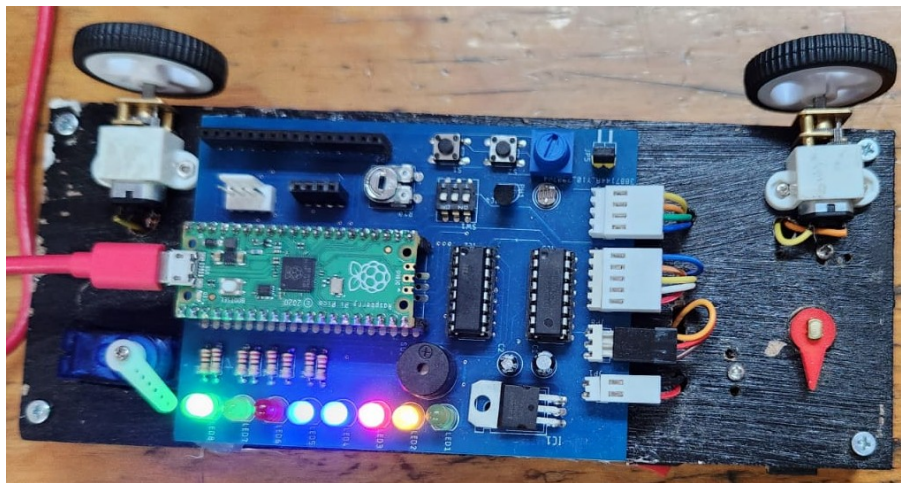


Figura 20: Actividad 9: Contador = 121 - 01111001

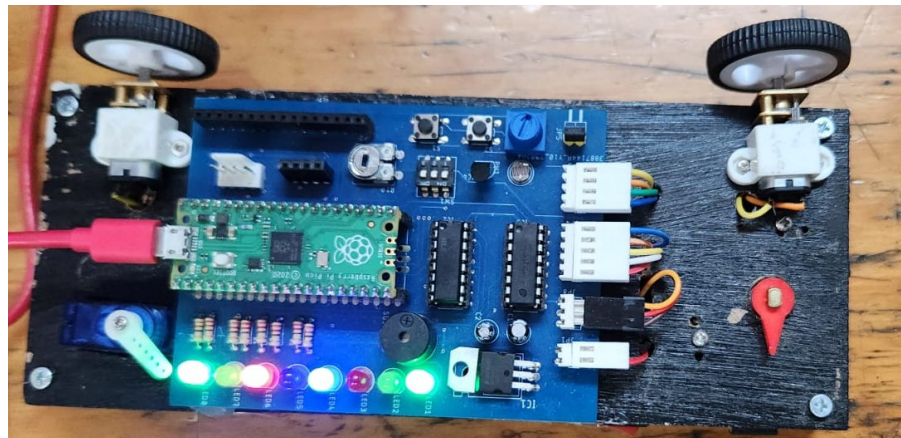


Figura 21: Actividad 9: Contador = 149 - 10010101

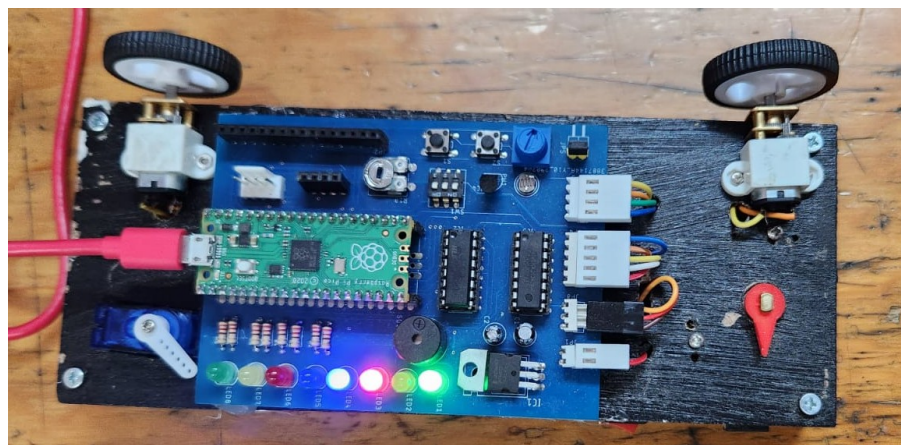


Figura 22: Actividad 9: Contador = 176 - 10110000

Análisis de resultados

A partir de las imágenes obtenidas se puede apreciar que la secuencia, en este caso un contador binario se implemento de forma correcta, en la descripción de cada imagen se puede observa el número en decimal y su equivalencia en número binario a través del encendido y apagado de los bits correspondientes para representar dicho número. Solo que debido a la distribución de los pines el led más a la izquierda, es decir el (LED 8) es el led menos significativo, mientras que el led más a la derecha (LED 1) es el led más representativo ya que si no se considera de esta manera no se estaría visualizando el contador de manera correcta.

Actividad 10

Usando las salidas correspondientes a los colores de los leds; realizar un programa que controle el funcionamiento de dos semáforos, de acuerdo a la siguiente tabla.

Estado	Salida	Requerimiento
1	V1, R2	5 segundos
2	V1, R2	5 intermitentes de 200 ms, en V1
3	A1, R2	3 segundos
4	R1, V2	5 segundos
5	R1, V2	5 intermitentes de 200 ms, en V2
6	R1, A2	3 segundos

Tabla 3-9. Control; actividad 10

Propuesta de solución

Para controlar los pines físicos del microcontrolador así como manejar los tiempos se debe de realizar la importación de las bibliotecas `machine` y `utime`. Como se van a manejar dos semáforos es necesario crear las variables individuales para cada luz de los semáforos (V1, A1, R1) y (V2, A2, R2) dichas variables se configuran como salidas debido a que el microcontrolador enviara una señal alta para encender los Leds respectivos.

Debido a que en un semáforo se deben tener luces contradictorias al mismo tiempo, es necesario al realizar un cambio que las luces se apaguen, es por esto que se implemento una función auxiliar `apagar_todos` en la cual al llamarla pone todos los leds con un valor de 0 apagando todos los leds, esto con la finalidad de limpiar las luces led.

Dado que se quiere que se repita la secuencia de luces de dos semáforos de forma indefinida, la lógica de la secuencia tiene que ir dentro de un ciclo `while` infinito.

Para implementar el funcionamiento descrito en la tabla, se optó por realizarlo por estados para una mejor comprensión del código donde en el estado 1 se llama a la función `apagar_todos` luego se enciende la luz verde del semáforo 1 (V1) y la luz roja del semáforo 2 (R2) para después esperar por 5[s], una vez transcurrido 5[s] se pasa al estado 2 donde por medio de un ciclo `for` de 5 iteraciones la luz V1 parpadee cinco veces, es decir un parpadeo por cada segundo, donde se apaga y prende cada 200 [ms].



Una vez ya haya parpadeado 5 veces se pasa al estado 3, donde se apagan todos los leds por medio de la función auxiliar, para posteriormente encender A1 y R2 esperando en este estado un total de 3[s]. Una vez ya transcurridos 3[s] se procede a apagar todos los leds, para encender R1 y V2 manteniendo estos valores por 5[s]

Después de esperar por 5[s] se pasa al estado 5, donde el rojo sigue encendido y se inicia otro ciclo `for` de 5 repeticiones donde la luz V2 se apaga y enciende cada 200[ms] indicando el efecto de parpadeo que la luz verde va cambiar. Después de que acaba el parpadeo de V2 se apagan todas las luces y se enciende R1 Y A2 pausando la ejecución por 3[s], al transcurrir 3[s] vuelve al estado inicial, es decir el estado 1.

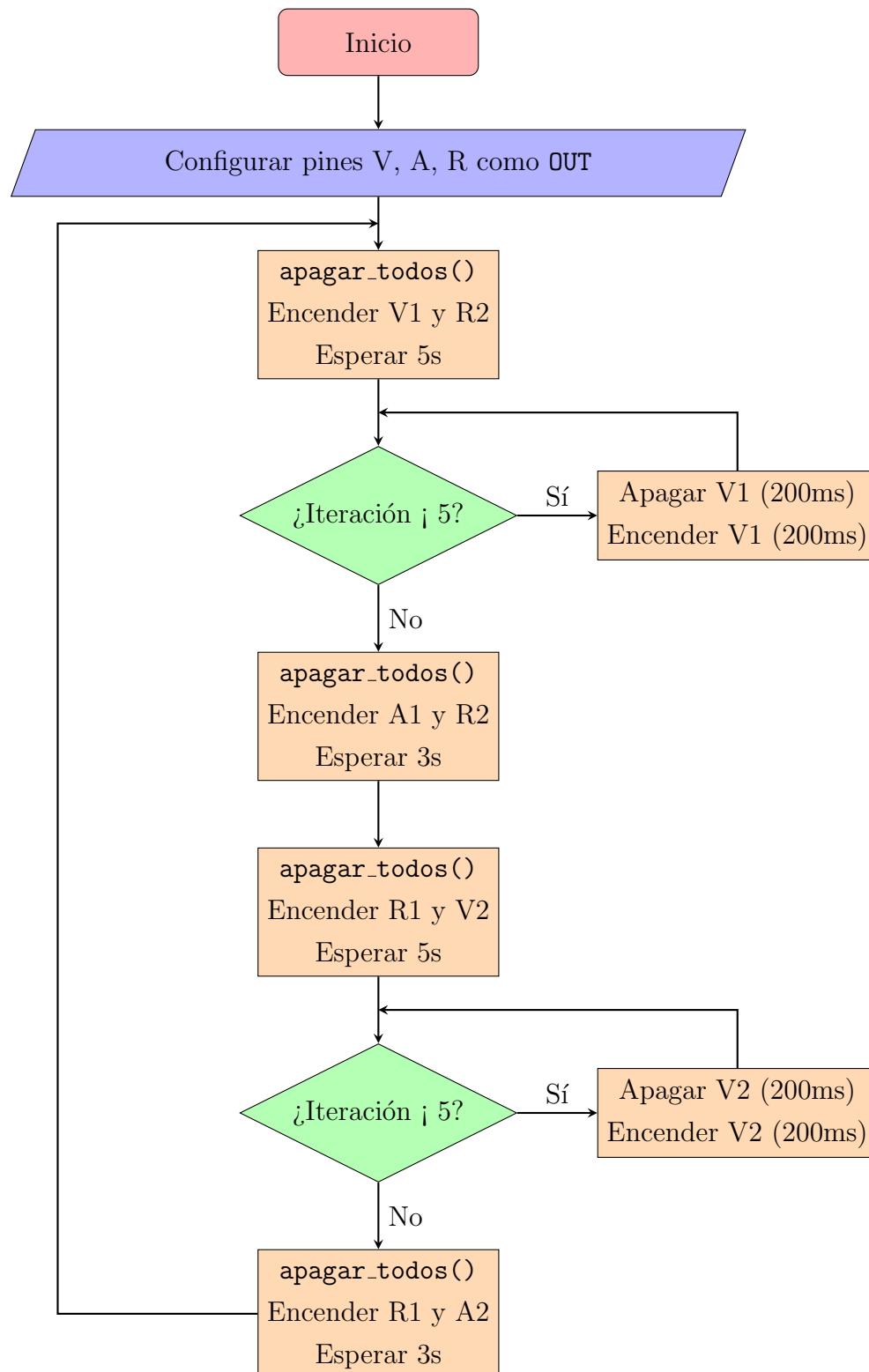


Diagrama de flujo: Control de 2 Semáforos

Desarrollo

Listing 10: Código de la Actividad 10

```
1 import machine
2 import utime
3
4 # Definición de pines para Semáforo 1
5 V1 = machine.Pin(0, machine.Pin.OUT) # Verde 1
6 A1 = machine.Pin(1, machine.Pin.OUT) # Amarillo 1
7 R1 = machine.Pin(2, machine.Pin.OUT) # Rojo 1
8
9 # Definición de pines para Semáforo 2
10 R2 = machine.Pin(5, machine.Pin.OUT) # Rojo 2
11 A2 = machine.Pin(6, machine.Pin.OUT) # Amarillo 2
12 V2 = machine.Pin(7, machine.Pin.OUT) # Verde 2
13
14 # Función auxiliar para apagar todos los LEDs de los semáforos
15 def apagar_todos():
16     V1.value(0)
17     A1.value(0)
18     R1.value(0)
19     V2.value(0)
20     A2.value(0)
21     R2.value(0)
22
23 while True:
24     apagar_todos()
25     # Estado 1: V1 encendido, R2 encendido por 5 segundos
26     V1.value(1)
27     R2.value(1)
28     utime.sleep_ms(5000)
29
30     # Estado 2: V1 intermitente (5 veces, 200 ms), R2 sigue
31     # encendido
32     for _ in range(5):
33         V1.value(0) # Apaga V1
```



```
33         utime.sleep_ms(200)    # Retardo 200 ms
34         V1.value(1)           # Enciende V1
35         utime.sleep_ms(200)    # Retardo 200 ms
36
37     apagar_todos()
38     # Estado 3: A1 encendido, R2 encendido por 3 segundos
39     A1.value(1)
40     R2.value(1)
41     utime.sleep_ms(3000)
42
43     apagar_todos()
44     # Estado 4: R1 encendido, V2 encendido por 5 segundos
45     R1.value(1)
46     V2.value(1)
47     utime.sleep_ms(5000)
48
49     # Estado 5: R1 sigue encendido, V2 intermitente (5 veces, 200 ms
50     )
51     for _ in range(5):
52         V2.value(0)           # Apaga V2
53         utime.sleep_ms(200)    # Retardo 200 ms
54         V2.value(1)           # Enciende V2
55         utime.sleep_ms(200)    # Retardo 200 ms
56
57     apagar_todos()
58     # Estado 6: R1 encendido, A2 encendido por 3 segundos
59     R1.value(1)
60     A2.value(1)
61     utime.sleep_ms(3000)
```

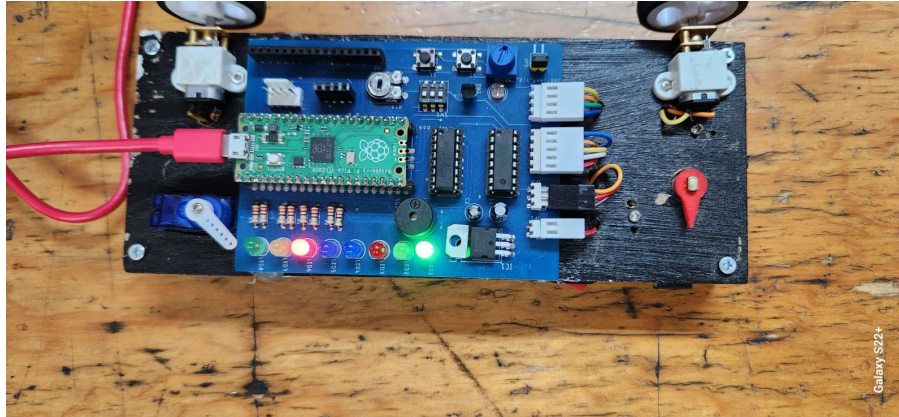


Figura 23: Actividad 10: Semáforo Estado 1

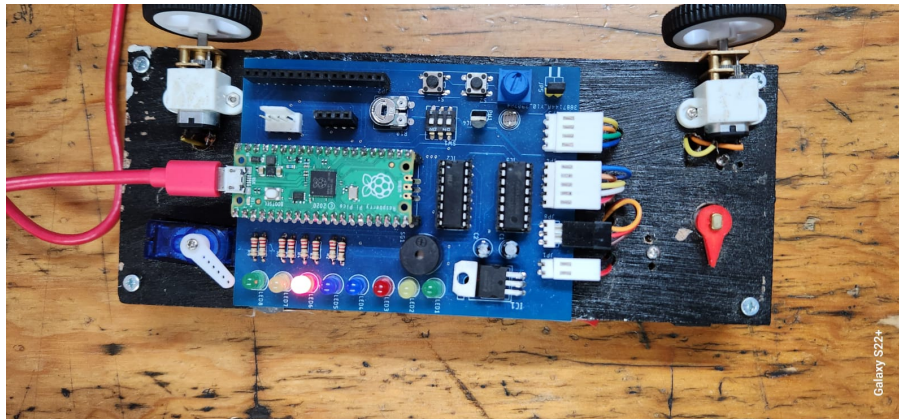


Figura 24: Actividad 10: Semáforo Estado 2

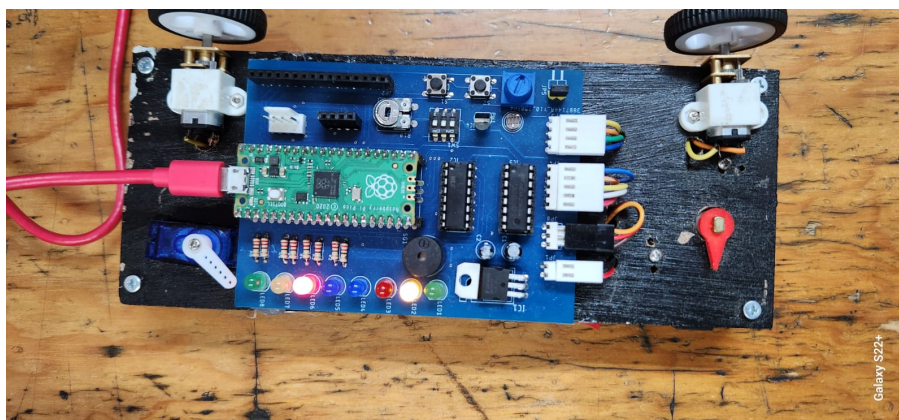


Figura 25: Actividad 10: Semáforo Estado 3

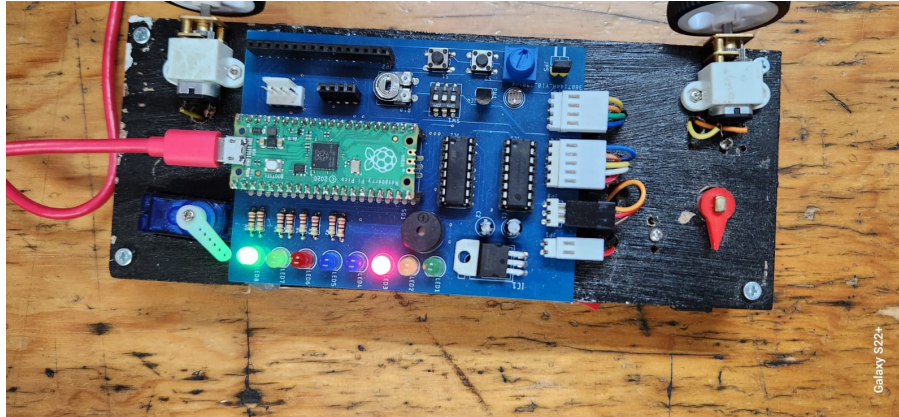


Figura 26: Actividad 10: Semáforo Estado 4

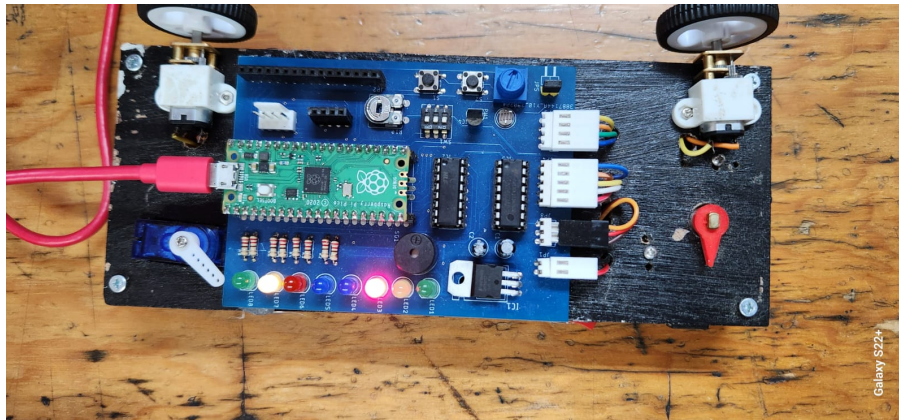


Figura 27: Actividad 10: Semáforo Estado 6

Análisis de resultados

Por medio de las imágenes tomadas se pueden observar la mayoría de los estados descritos en la solución del problema, de forma que cada que pasa el tiempo especificado en la tabla se pasa al siguiente estado con las luces correspondientes en cada estado. De manera que se implemento de forma correcta el comportamiento de los dos semáforos donde por medio de las instrucciones de encendido y apagado mandando los valores lógicos de 0 y 1 así como por medio de la función `sleep` realizar pausas en cada estado así como estructuras de repetición se puede implementar el sistema de dos semáforos.

2. Conclusiones:

- **Espinoza Matamoros Percival Ulises:** Gracias a la realización de la práctica pude aprender y comprender el funcionamiento básico de los pines GPIO de la Raspberry Pi Pico 2 usando Micropyton y el entorno de desarrollo Thony, donde por medio de la librería `machine` se puede interactuar con el hardware configurando si los pines se van a comportar como pines de entrada o salida, en este caso se configuraron como pines de salida, de forma que cada que se quiere encender un led se manda a ese pin un valor lógico de 1 mientras que si se desea apagar se debe de enviar un valor lógico de 0.

Durante la realización de las diversas actividades se comprendió como por medio de estas instrucciones así como lógica de programación como ciclos `for` se pueden implementar secuencias complejas desde el parpadeo de una secuencia en específico de leds, contadores hasta secuencia simultánea de dos semáforos. Comprendiendo así la flexibilidad y facilidad que ofrece MicroPython para implementar programas.

- **Flores Colin Victor Jaziel:** La realización de esta práctica número 7 me permitió consolidar el dominio de la plataforma Raspberry Pi Pico y su microcontrolador RP2040. A través del entorno Thonny y el lenguaje MicroPython que es una herramienta sumamente sencilla, se logró la transición exitosa desde el control básico de un solo bit (parpadeo de un LED) hasta la gestión compleja de buses de datos de 8 bits y sistemas de control secuencial con lógica de semaforización.

Concluyo que el uso de estructuras de datos como listas y bucles anidados optimiza significativamente la interacción con las terminales GPIO, permitiendo desarrollar algoritmos modulares y escalables y la precisión en los retardos y la manipulación de estados lógicos mediante máscaras de bits demostraron que la plataforma es altamente eficiente para aplicaciones de control en tiempo real, cumpliendo satisfactoriamente con los objetivos de diseño y análisis propuestos en el laboratorio.

- **Lara Hernandez Angel Husiel:** Las instrucciones de alto nivel en Python se traducen en control directo sobre los pines GPIO, pudiendo manipular voltajes y estados lógicos con métodos tan simples como `value(1)` y `value(0)`, lo que reduce drásticamente la complejidad comparado con programar a nivel de registros.

Implementar secuencias como el efecto corredor, el encendido acumulativo y el contador binario de 8 bits me sirvió mucho para entender cómo los bucles `for` y `while` se com-



binan con las funciones de temporización para generar patrones precisos en hardware. También vi que usar operaciones bit a bit para extraer cada bit del contador y asignarlo a un GPIO específico es un recurso muy eficiente para representar valores numéricos en hardware. En resumen, esta práctica me dio bases muy buenas para entender cómo se programa un microcontrolador de forma estructurada, aprovechando las ventajas de un lenguaje interpretado sin perder el control fino sobre el hardware.

Referencias

- Arm Ltd. (2020). *Arm Cortex-M0+ Processor Datasheet* (Datasheet) (Documento PDF. Año aproximado). Arm Limited.
- Gómez, A. (2015, agosto). *Resistencia pull up y pull down con Arduino, para qué sirven*. Consultado el 27 de febrero de 2026, desde <https://programarfacil.com/blog/arduino-blog/resistencia-pull-up-y-pull-down/> Blog de tecnología y programación.
- Raspberry Pi Ltd. (2025, 20 de febrero). *RP2040 Datasheet: A microcontroller by Raspberry Pi*. Ver. build-version: 3184e62-clean. Consultado el 27 de febrero de 2026, desde <https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf>
- Raspberry Pi Ltd. (2026). *Microcontroller chips - Raspberry Pi Documentation*. Consultado el 26 de febrero de 2026, desde <https://www.raspberrypi.com/documentation/microcontrollers/microcontroller-chips.html>
- Raspberry Pi Stack Exchange Users. (2021, diciembre). *Why is a resistor needed for LEDs?* Consultado el 27 de febrero de 2026, desde <https://raspberrypi.stackexchange.com/questions/4357/why-is-a-resistor-needed-for-leds> Foro de Q&A. Colaboradores: Steve Robillard, joan, Jivad.
- Transfer Multisort Elektronik (TME). (2024). *GPIO - definición — Glosario TME*. Consultado el 27 de febrero de 2026, desde <https://www.tme.com/mx/es/news/library-articles/glossary/page/61888/gpio-general-purpose-inputoutput-definicion/> Artículo de glosario técnico.